Validation, error types and error trapping

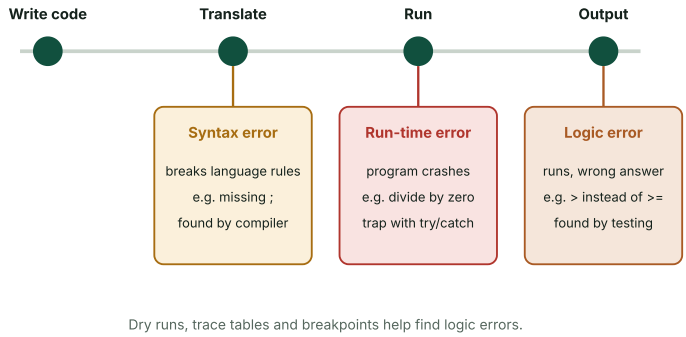
D21 FACT FILE

Specification: students should be able to

- Data validation, including presence, length, type and format checks
- Detection and correction techniques for syntax, execution and logic errors
- Simple error trapping techniques

CCEA fact file: Error Handling

Key words: validation, GIGO, presence check, length check, type check, format check, range check, validation loop, syntax error, execution error, run-time error, logic error, debugging, breakpoint, error trapping, exception, try/catch



When syntax, run-time and logic errors appear during development

Validation

Validation checks that data is reasonable before it is processed: presence (not blank), length (number of characters), type (correct data type), format (correct pattern) and range (within limits). It reduces input errors but cannot prove data is correct. A validation loop repeatedly asks for input while the data is invalid, showing an error message each time.

Errors

Syntax errors break the language's rules and are reported by the translator before the program can run. Execution (run-time) errors happen while the program runs and cause it to crash, e.g. division by zero or opening a missing file. Logic errors let the program run but produce wrong results; they are found by testing against expected outcomes, dry runs, trace tables and debugging tools such as breakpoints.

Error trapping

Error trapping (exception handling) uses try ... catch in C# to catch run-time errors such as FormatException and DivideByZeroException and respond with a helpful message instead of crashing, making the program more robust.

T Key terms from the mark schemes

Syntax error
Breaking the grammar rules of the language; found by the translator before running.
Credited: U4 2019 Q5, 2023 Q4, 2025 Q3

Logic error
The program runs but gives incorrect results because the reasoning is wrong.
Credited: U4 2019 Q5, 2024 Q6, 2025 Q3

Execution / run-time error
An error that occurs while the program runs and causes it to crash, e.g. divide by zero.
Credited: U4 2022 Q9, 2024 Q6

Error trapping
Code that detects a run-time error and handles it instead of crashing, e.g. try/except.
Credited: U4 2022 Q10

C C# example

Validation loops

```
Console.Write("Frames (10-40): ");
int frames = Convert.ToInt32(Console.ReadLine());
while (frames < 10 || frames > 40)
{
    Console.WriteLine("Error: enter a number from 10 to 40");
    Console.Write("Frames (10-40): ");
    frames = Convert.ToInt32(Console.ReadLine());
}
Console.WriteLine("Accepted");
```

Exam focus

Validation: 2019 Q1(b), 2022 Q4(b), 2023 Q4(d) and Q5(a), 2024 Q3(b)–(c), 2025 Q6(d). Errors: 2019 Q5, 2022 Q9(b), 2023 Q4(c), 2024 Q6 (QWC), 2025 Q3(c)(ii). Error trapping: 2022 Q10(b).

Unit 4 2025 Q6	[25]	Tent hire algorithm; Boolean; constants; validation true/false; Y/N validation loop
Unit 4 2025 Q3	[15]	Musician hire algorithm; variables and data types; IDE aids; syntax vs logic
Unit 4 2024 Q6	[6]	Execution and logic errors and handling (QWC)
Unit 4 2024 Q3	[17]	Travel insurance cost algorithm; validation; real data type; WHILE range-check loop
Unit 4 2023 Q5	[8]	Length, presence and format checks; string functions for a password

? Check yourself

1. Explain why validation cannot guarantee that data is correct. [2]
2. Describe a format check and give an example in a password system. [2]
3. Write pseudocode that repeatedly asks for 'Y' or 'N' until one of them is entered, showing an error message for invalid input. [5]

Answers: 1. Validation only checks that data is sensible/follows set rules (e.g. within a range) (1) / data can be valid but still wrong, e.g. an age of 21 typed instead of 22 passes a range check (1) 2. A format check ensures the data matches a set pattern (1) / e.g. the password must end with a symbol, or contain '@' (1) 3. INPUT answer

```
WHILE answer <> "Y" AND answer <> "N"
   OUTPUT "Error: please enter Y or N"
   INPUT answer
END WHILE
```

Marks: initial input (1) / condition-controlled loop (WHILE or REPEAT...UNTIL) (1) / correct condition using AND (or UNTIL answer = "Y" OR answer = "N") (1) / error message shown only for invalid input, inside the loop (1) / input again inside the loop (1). Accept equivalent C#.